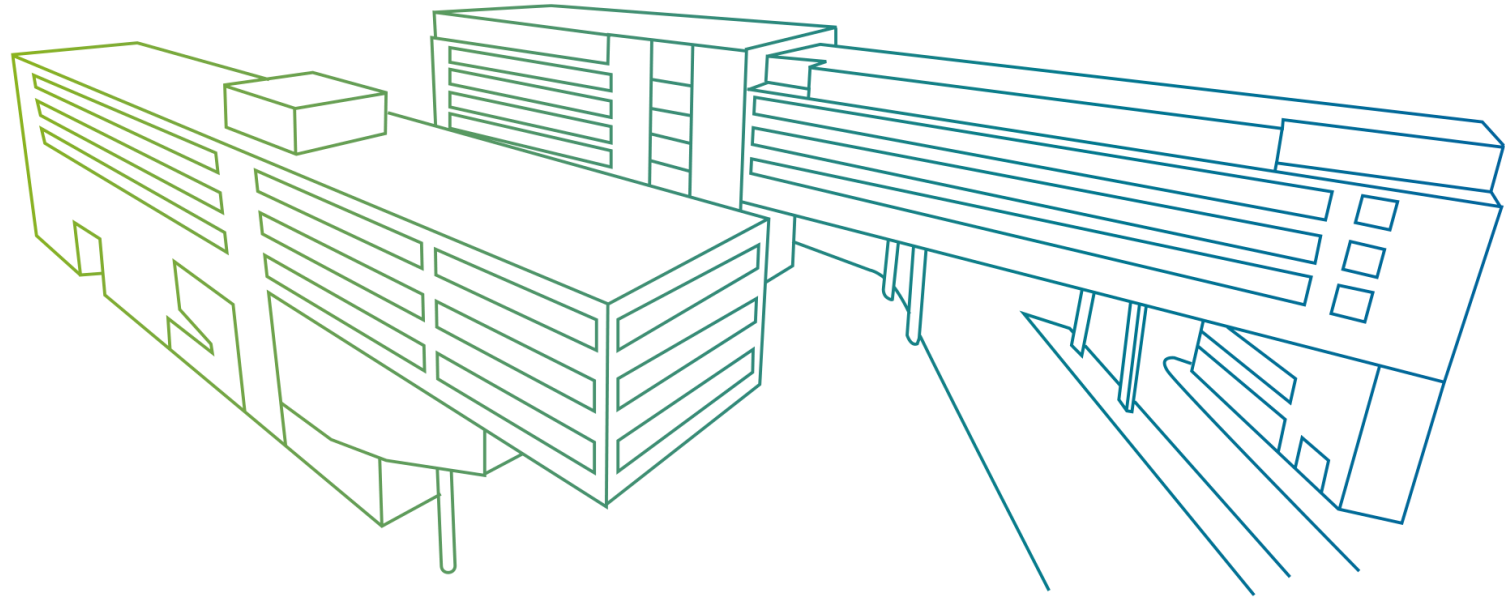




Probabilistic Robotics

Kalman Filter & Extended Kalman Filter

Lucas Muster, MSc

HW: Calculation Door Sensor

Door Robot (Exercise)

Page Settings More ▾

This mobile robot was developed to open/close doors. However, the developers did an awful job, and implemented a rather bad door sensor. However, our task is to implement a reliable solution. Hence, we must infer the state of the door. We need the following data:

We do not the state of the door at $t = 0$. Since we haven't gathered any sensor data yet, $bel(x_0 = \text{Open}) = bel(x_0 = \text{Closed}) = 0.5$. Note, that the state is a binary variable. The measurement model is

$$p(z_t = \text{Open} | x_t = \text{Open}) = 0.6$$

$$p(z_t = \text{Closed} | x_t = \text{Open}) = 0.4 \text{ (**Remember**: one of the key properties of this "function" p was that the sum is 1)}$$

$$p(z_t = \text{Open} | x_t = \text{Closed}) = 0.2$$

$$p(z_t = \text{Closed} | x_t = \text{Closed}) = 0.8 \text{ (A rather bad sensor...)}$$

The "motion model" in this particular case describes the outcome of a command (push the door or do nothing). The term "motion" in this case means a motion in the state. Hence, if we want to investigate the outcome of a command:

$$p(x_t = \text{Open} | u_t = \text{push}, x_{t-1} = \text{Open}) = 1 \text{ (The door is open if we want to open an open door)}$$

$$p(x_t = \text{Closed} | u_t = \text{push}, x_{t-1} = \text{Open}) = 0 \text{ (The door will not be closed if we open an opened door)}$$

$$p(x_t = \text{Open} | u_t = \text{push}, x_{t-1} = \text{Closed}) = 0.8 \text{ (The robot can open a closed door with the probability of 80\%)}$$

$$p(x_t = \text{Closed} | u_t = \text{push}, x_{t-1} = \text{Closed}) = 0.2 \text{ (The robot fails to open a closed door with the probability of 20\%)}$$

... and in case of "do nothing":

$$p(x_t = \text{Open} | u_t = \text{doNothing}, x_{t-1} = \text{Open}) = 1$$

$$p(x_t = \text{Closed} | u_t = \text{doNothing}, x_{t-1} = \text{Open}) = 0$$

$$p(x_t = \text{Open} | u_t = \text{doNothing}, x_{t-1} = \text{Closed}) = 0$$

$$p(x_t = \text{Closed} | u_t = \text{doNothing}, x_{t-1} = \text{Closed}) = 1$$

Your Task

Calculate $\overline{bel}(x_t)$ and $bel(x_t)$ für $u_t = \text{doNothing}$. You receive a "door is open" signal from the sensor at $t=5$.

In order to solve this riddle, please do the following tasks and answer the questions below:

- Which PDF is used to describe the door sensor?
- Use the equations from the slides to calculate $\overline{bel}(x_t)$ and $bel(x_t)$

Overview

- Kalman Filter:
 - Estimating states in linear Gaussian systems.
 - Predict and update steps for state estimation.
- Extended Kalman Filter (EKF):
 - Handling non-linear systems with Taylor approximation.
 - EKF for localization and sensor fusion.

Recap: The Bayes Filter

This results in a two-step approach

- Step 1: Prediction
 - We rely on the previous state
 - The robot is "moved" using the kinematic model and the command
 - We solve the integral, we get the prediction

The prediction (the conditional probability) is shortened to $\overline{bel}(\vec{x}_t)$

Recap: The Bayes Filter

- Step 1: Prediction
- Step 2: Correction
 - Evaluation of prediction.
 - Correction of prediction using a measurement.
 - The correction (the conditional probability) is shortened using $bel(\vec{x}_t)$.

Recap: The Bayes Filter

- Merging $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$ and $p(\vec{z}_t | \vec{x}_t)$
- Two-stage algorithm: Movement prediction and correction.
- Approach:

Iterative algorithm: $\vec{x}_t \rightarrow \vec{x}_{t+1}$

Calculation using $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$

Correction using $p(\vec{z}_t | \vec{x}_t)$

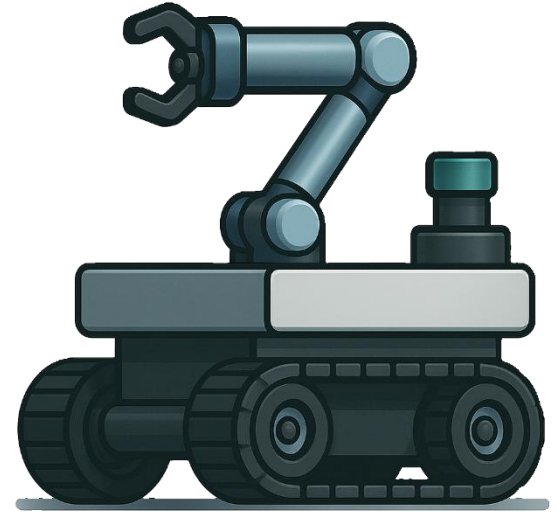
Theoretical approach: Bayes Filter

```
1:   Algorithm Bayes_filter( $bel(x_{t-1}), u_t, z_t$ ):  
2:     for all  $x_t$  do  
3:        $\overline{bel}(x_t) = \int p(x_t | u_t, x_{t-1}) bel(x_{t-1}) dx$   
4:        $bel(x_t) = \eta p(z_t | x_t) \overline{bel}(x_t)$   
5:     endfor  
6:     return  $bel(x_t)$ 
```

Figure: Bayes Filter Localization (see page 24 in (Thrun, 2006))

Recap: Inaccuracy

- Problem:
 - We have a system with (a lot of) uncertainty
 - Sensor System and Movement
- Example:
 - Our mobile robot is driving forward
 - Wheel odometry
 - GPS Sensor



Core problem

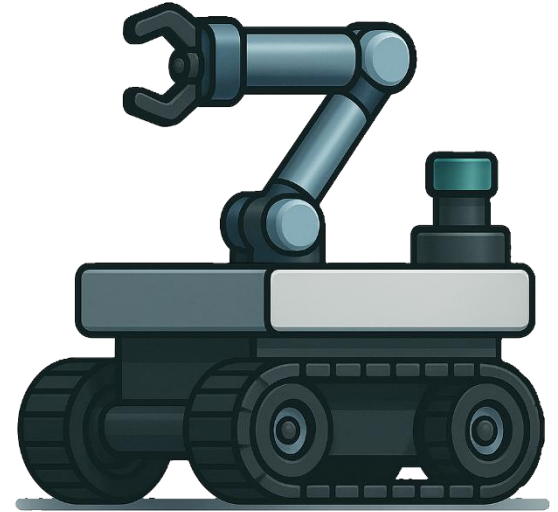
Two uncertain pieces of information :

1. Motion Model

- Based on physics
- Drift

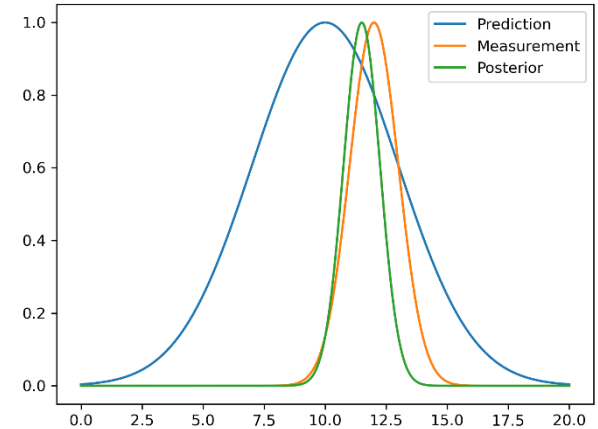
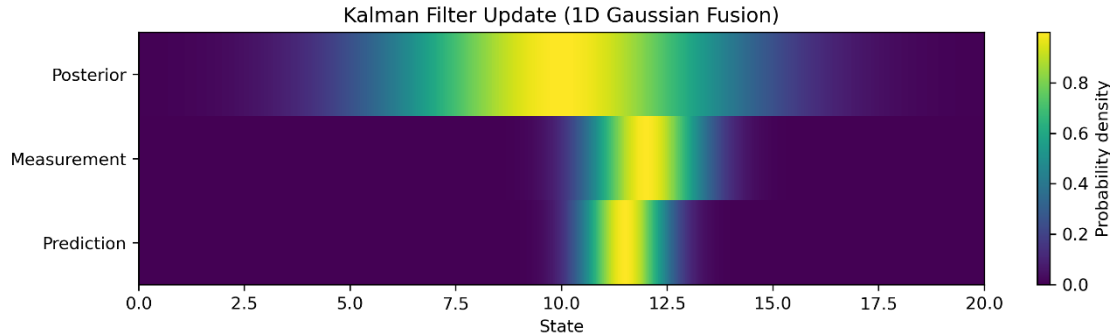
2. Sensor

- Measures directly
- Noise



Unsicherheit als Verteilung

- Belief = Probability
- We start with:
 - Our uncertainty is Gaussian
- What happens when we combine them?



Intuition des Kalman Gains

- **Fall 1: Bad sensor**

- big variance → trust motion

- **Fall 2: Sensor gut**

- small variance → trust sensor

The Kalman filter is a weighted average, but one that is weighted intelligently.

$$estimate = w1 \cdot prediction + w2 \cdot measurement$$

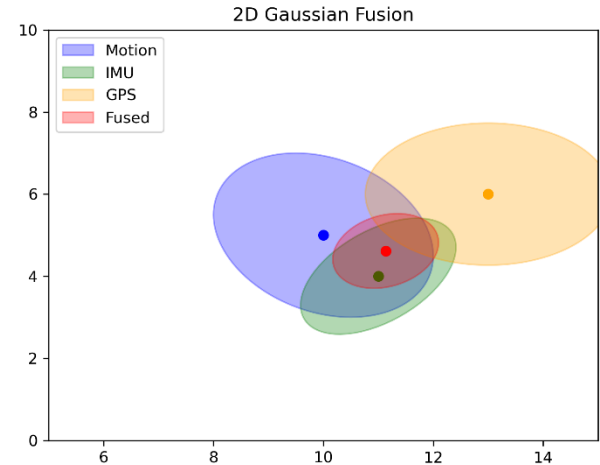
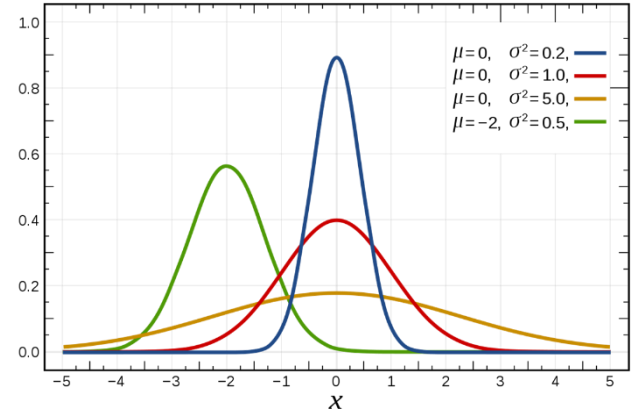
→ Weights depend on uncertainty

- Prediction: $x = 10$, high uncertainty
- Measurement: $z = 12$, low uncertainty
- Which is the best estimate?

Variance = Uncertainty

- A state estimate is not a single value, but a distribution
- The variance describes how uncertain we are
- Large variance → wide Gaussian
- Small variance → narrow Gaussian

$$\Sigma = \begin{bmatrix} \sigma^2_x & cov_{xy} \\ cov_{yx} & \sigma^2_y \end{bmatrix}$$



Exercise

- A robot estimates its position using three different sources:
 - Motion model (prediction)
 - IMU (short-term accurate, but noisy)
 - GPS (absolute position, but less precise)
- Each source provides an estimate with an uncertainty estimate.
- **Given:**
 - Motion model: $\mu = 10, \sigma^2 = 4$
 - IMU: $\mu = 11, \sigma^2 = 2$
 - GPS: $\mu = 13, \sigma^2 = 3$
- **Tasks:**
 - Implement a function that fuses multiple Gaussian estimates.
 - Compute the fused mean and variance.
 - Print/Plot the result(s).

From Bayes Filter to the Kalman Filter

- $bel(\vec{x}_t) \sim N(\vec{\mu}_t | \Sigma_t)$: The robots state at timeslot t has an expectation of $\vec{\mu}_t$ and a covariance of Σ_t
- $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$: A robots motion at timeslot t based on $bel(\vec{x}_{t-1})$ described by a Gaussian.
- $p(\vec{z}_t | \vec{x}_t)$: A evaluation of a pseudo measurement and the real measurement

```
1:   Algorithm Bayes_filter( $bel(x_{t-1}), u_t, z_t$ ):  
2:     for all  $x_t$  do  
3:        $\overline{bel}(x_t) = \int p(x_t | u_t, x_{t-1}) bel(x_{t-1}) dx$   
4:        $bel(x_t) = \eta p(z_t | x_t) \overline{bel}(x_t)$   
5:     endfor  
6:     return  $bel(x_t)$ 
```

Figure: Bayes Filter Localization (see page 24 in (Thrun, 2006))

From Bayes Filter to the Kalman Filter

- The solution of the integral in line 3: $\overline{bel}(\vec{x}_t)$ is a Gaussian.
 - The solution of the multiplication in line 4: $bel(\vec{x}_t)$ is a Gaussian as well.
- we 'call' a Kalman filter function with the previous estimation, a measurement and a command and get a Gaussian containing the new estimation.

However, it is not that simple:

- How do we get from an input Gaussian to an output Gaussian?
- How can we calculate the Gaussian's parameter ($\vec{\mu}_t$ and Σ_t)?

Understanding Kalman Filter

- Assumption: multivariate gaussian distribution (see cheat sheet).

$$p(\vec{x}_t) = \det(2\pi R)^{\frac{1}{2}} * e^{(-\frac{1}{2}(\vec{x}_t - \vec{\mu}_t)^T R^{-1}(\vec{x}_t - \vec{\mu}_t))}$$

- Transition is a linear movement with gaussian noise.
- Basis:

$$\vec{x}_t = A\vec{x}_{t-1} + B\vec{u}_t + \epsilon_t \text{ where } \epsilon_t \text{ is gaussian noise}$$

$$p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t) = \det(2\pi R)^{-\frac{1}{2}} * e^{(-\frac{1}{2}(\vec{x}_t - A\vec{x}_{t-1} - B\vec{u}_t)^T R^{-1}(\vec{x}_t - A\vec{x}_{t-1} - B\vec{u}_t))}$$

The Kalman Filter

- State transition $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$ is linear

$$\vec{\mu}_t = A * \vec{\mu}_{t-1} + B * \vec{u}_t + \epsilon_t$$

- Linear: $\vec{\mu}_t$ is a linear combination of $\vec{\mu}_{t-1}$
- Exercise:

$$\vec{\mu}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = ?$$

The Kalman Filter

$$\vec{\mu}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta \theta \end{bmatrix} + \epsilon_t$$

The Kalman Filter

- State transition $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$ is linear

$$\vec{\mu}_t = A * \vec{\mu}_{t-1} + B * \vec{u}_t + \epsilon_t$$

- Linear: $\vec{\mu}_t$ is a linear combination of $\vec{\mu}_{t-1}$
- Exercise:

$$\vec{\mu}_t = \begin{bmatrix} x_t \\ \dot{x}_t \\ y_t \\ \dot{y}_t \end{bmatrix} = ?$$

The Kalman Filter

$$\vec{\mu}_t = \begin{bmatrix} x_t \\ \dot{x}_t \\ y_t \\ \dot{y}_t \end{bmatrix} = \begin{bmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} x_{t-1} \\ \dot{x}_{t-1} \\ y_{t-1} \\ \dot{y}_{t-1} \end{bmatrix} + \begin{bmatrix} \frac{1}{2} \Delta t^2 & 0 \\ \Delta t & 0 \\ 0 & \frac{1}{2} \Delta t^2 \\ 0 & \Delta t \end{bmatrix} * \begin{bmatrix} a_x \\ a_y \end{bmatrix} + \epsilon_t$$

1:	Algorithm Kalman_filter ($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
2:	$\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
3:	$\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$

Understanding Kalman Filter: Measurement

- We want to evaluate $\vec{z}_t$ relying on the prediction
- We need the measurement matrix C , which maps from the state to a pseudo measurement
- The error of measurement is Gaussian distributed
- Basis:

$$\vec{z}_t = C\vec{x}_t + \delta_t \text{ where } \delta_t \text{ is gaussian noise}$$

$$p(\vec{z}_t | \vec{x}_t) = \det(2\pi Q)^{-\frac{1}{2}} * e^{(-\frac{1}{2}(\vec{z}_t - C\vec{x}_t)^T Q^{-1}(\vec{z}_t - C\vec{x}_t))}$$

Understanding Kalman Filter: Measurement

- In case of the IMU sensor:
 - Acceleration in x, y and the angular acceleration around the z - axis
- But our state is in the world coordinate system
- We have to rotate the IMU into the world coordinate system
 - Watch out: we are again getting a non linear system
- For the Kalman filter with the state $\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$ and a linear system we could use only θ
- $\vec{z}_t = C\vec{x}_t + \delta_t$ where δ_t is gaussian noise
- How does C has to look like here?

Understanding Kalman Filter: Measurement

- Who can be trusted more? Prediction or measurement?
- We know the prediction's uncertainty (see $bel(\vec{x}_t)$)
- We do know the measurement noise Q (we can calibrate that)
- The Kalman gain K_t tries to calculate a "trust"-value based on those uncertainties

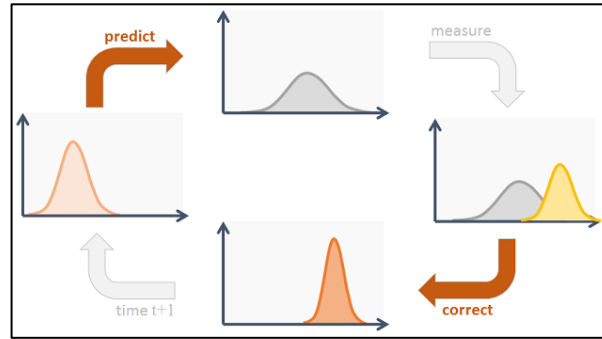
$$4: \quad K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$$

Understanding Kalman Filter: Measurement

- The Kalman Gain determines how much we trust:
 - the **prediction** (model)
 - the **measurement** (sensor)
- Interpretation
 - Large $K_t \rightarrow$ trust measurement more
 - Small $K_t \rightarrow$ trust prediction more
- K_t is computed automatically based on uncertainty:
 - high measurement noise $\rightarrow$ small K_t
 - high prediction uncertainty $\rightarrow$ large K_t

Understanding Kalman Filter

- We discussed the motion and sensor models
- What happened to the state?
 - The expectation $\vec{\mu}_t$ and covariance after applying the motion model are calculated
 - "Evaluation" of prediction relying on the measurement matrix C



The motion is "uncertain": The covariance will "grow"

The sensor observes same physical action. Based on sensor noise, we can update the prediction.

Kalman Filter Algorithm: Prediction

- Line 2: Apply the motion model
- Line 3: Get prediction of covariance
- Line 4: Calculates the Kalman gain K_t

Analyze the equation carefully: Note the ratio of the "converted" covariance of the prediction and the sensor covariance. Without any linear algebra we see that high measurement noise will reduce the numbers of the Kalman gain.

```
1:  Algorithm Kalman filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:       $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$   
3:       $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$   
4:       $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$   
5:       $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$   
6:       $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$   
7:      return  $\mu_t, \Sigma_t$ 
```

Figure: The Kalman filter (Thrun, 2006)

Kalman Filter Algorithm: Correction

- Line 5: Correction of expectation using Kalman gain
- Line 6: Correction of covariance matrix
- Line 7: Returns corrected belief

```
1:  Algorithm Kalman filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:       $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$   
3:       $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$   
4:       $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$   
5:       $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$   
6:       $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$   
7:      return  $\mu_t, \Sigma_t$ 
```

Figure: The Kalman filter (Thrun, 2006)

Exercise: Kalman Filter

- Implement a simple Kalman Filter for robot pose estimation
- Think of any linear system with state $\vec{\mu}_t$
- Use a simple motion model and IMU measurements.
- Simulate some data (Motion and measurements)

- Implement a Kalman filter
 - Prediction step (2 and 3)
 - Compute the Kalman Gain (4)
 - Implement the update step (5 and 6)

```
1: Algorithm Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:    $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$   
3:    $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$   
4:    $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$   
5:    $\mu_t = \bar{\mu}_t + K_t (z_t - C_t \bar{\mu}_t)$   
6:    $\Sigma_t = (I - K_t C_t) \bar{\Sigma}_t$   
7:   return  $\mu_t, \Sigma_t$ 
```

Figure: The Kalman filter (Thrun, 2006)

- Visualization of the true data, the estimated data and the IMU measurement
- Plot the Kalman Gain over time

Restrictions of the Kalman Filter

- Linear assumption problem
 - State transition: A is a simplification (relies on a linear model)
 - Measurement matrix: C is a simplification
 - Example: Robot drives circle-like
- Gaussian assumption problem
 - Gaussian noise of measurement is a simplification

We do not have a linear transition

- The transition is not a linear model: a robot moves in circular pattern
- How to overcome this: we linearize the nonlinear function!
- This is the extended Kalman filter (EKF)

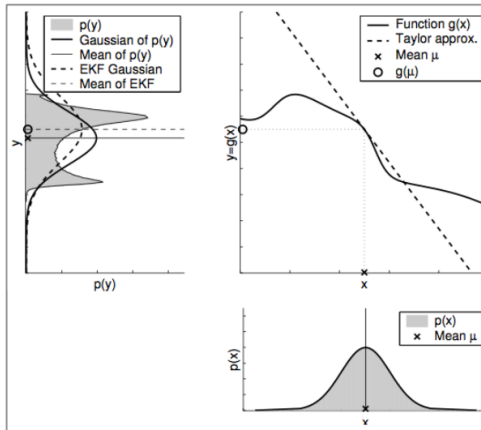


Figure: Visualization of a nonlinear transition (Thrun, 2006) - e.g.: $\vec{x}_{t-1}$ to $\vec{x}_t$

The Extended Kalman Filter

- The motion is implemented using $A * \vec{\mu}_{t-1} + B * \vec{u}_t$
- But for a differential driven robot we calculate:

$$\begin{bmatrix} X_t \\ Y_t \\ \theta_t \end{bmatrix} = \begin{bmatrix} X_{t-1} \\ Y_{t-1} \\ \theta_{t-1} \end{bmatrix} + \begin{bmatrix} \delta d * \cos(\theta_{t-1} + \delta\theta) \\ \delta d * \sin(\theta_{t-1} + \delta\theta) \\ \delta\theta \end{bmatrix} \text{ with } \delta d = \frac{(\Delta\phi_l + \Delta\phi_r) * r}{2} \text{ and } \delta\theta = \frac{(\Delta\phi_l + \Delta\phi_r) * r}{b}$$

- So, we know, the linear model will not fit well

The Extended Kalman Filter

- EKF uses Taylor expansion.
- Linearization for "non-linear" transitions.
 - Allows "more" non-linear systems
 - Step closer to "real" systems
 - Still single guess using $\{\vec{\mu}_t, \Sigma_t\}$

The Extended Kalman Filter

- EKF uses Taylor expansion.
- Linearization for "non-linear" transitions.
 - Allows "more" non-linear systems
 - Step closer to "real" systems
 - Still single guess using $\{\vec{\mu}_t, \Sigma_t\}$
- Introduction of non-linear functions
 - $g(.)$ for state transition
 - $h(.)$ for measurement
- Comparable to Kalman filter
 - A, B : replaced by $g(.)$
 - C : replaced by $h(.)$

$$\vec{x}_t = g(\vec{u}_t, \vec{x}_{t-1}) + \epsilon_t$$

$$\vec{z}_t = h(\vec{x}_t) + \delta_t$$

EKF: New Challenge

- Due to non-linear functions: No longer closed form solution
- Kalman Filter: Exact calculation of belief
- New Task: Estimate $\{\vec{\mu}_t, \Sigma_t\}$ efficiently
- New formulation of $p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t)$ and $p(\vec{z}_t | \vec{x}_t)$

EKF: State Transition Probability

- Because of $g(\cdot)$, no closed form for gaussian distribution
- Linearization using tailor expansion:

$$Tf(x; a) = f(a) + f'(a) * (x - a) + \frac{f''(a)}{2} * (x - a)^2 + \frac{f'''(a)}{6} * (x - a)^3 + \dots$$

$$Tf(x; a) = \sum_{n=0}^{\infty} \frac{f^{(n)}}{n!} * (x - a)^n$$

- Definition for first order tailer expansion:

$$T_1f(x; a) \approx f(a) + f'(a) * (x - a)$$

EKF: State Transition Probability

- Due to that:

$$g(\vec{u}_t, \vec{x}_{t-1}) \approx g(\vec{u}_t, \vec{\mu}_{t-1}) + g'(\vec{u}_t, \vec{\mu}_{t-1})(\vec{x}_{t-1} - \vec{\mu}_{t-1}) = g(\vec{u}_t, \vec{\mu}_{t-1}) + G_t(\vec{x}_{t-1} - \vec{\mu}_{t-1})$$

- $G_t = \frac{\partial g}{\partial x} \big|_{\vec{\mu}_{t-1}}$ is the Jacobian
- Now we can write an approximation for state transition probability

$$p(\vec{x}_t | \vec{x}_{t-1}, \vec{u}_t) \approx \det(2\pi R)^{-\frac{1}{2}} * e^{(-\frac{1}{2}(\vec{x}_t - g(\vec{u}_t, \vec{\mu}_{t-1}) - G_t(\vec{x}_{t-1} - \vec{\mu}_{t-1}))^T R^{-1}(\vec{x}_t - g(\vec{u}_t, \vec{\mu}_{t-1}) - G_t(\vec{x}_{t-1} - \vec{\mu}_{t-1})))}$$

- Prediction approximation: $g(\vec{u}_t, \vec{\mu}_{t-1}) + G_t(\vec{x}_{t-1} - \vec{\mu}_{t-1})$

EKF: Nonlinear function $g(\cdot)$

- For the state:

$$\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} \quad \text{and} \quad \vec{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$$

EKF: Nonlinear function $g(\cdot)$

- For the state:

$$\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix} \quad \text{and} \quad \vec{u}_t = \begin{bmatrix} v_t \\ \omega_t \end{bmatrix}$$

- For one time step:

$$\begin{aligned} x_t &= x_{t-1} + v_t * \cos(\theta_{t-1}) \Delta t \\ y_t &= y_{t-1} + v_t * \sin(\theta_{t-1}) \Delta t \\ \theta_t &= \theta_{t-1} + \omega_t \Delta t \end{aligned}$$

- Or in vector form:

$$g(u_t, x_{t-1}) = \begin{bmatrix} x_{t-1} + v_t * \cos(\theta_{t-1}) \Delta t \\ y_{t-1} + v_t * \sin(\theta_{t-1}) \Delta t \\ \theta_t = \theta_{t-1} + \omega_t \Delta t \end{bmatrix}$$

EKF: Jacobian G

- $G_t = \frac{\partial g}{\partial x} \big|_{\vec{\mu}_{t-1}}$ with state $\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$

EKF: Jacobian G

- $G_t = \frac{\partial g}{\partial x} \big|_{\vec{\mu}_{t-1}}$ with state $\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$

$$g_1 = x_{t-1} + v_t * \cos(\theta_{t-1}) \Delta t$$

$$g_2 = y_{t-1} + v_t * \sin(\theta_{t-1}) \Delta t$$

$$g_3 = \theta_{t-1} + \omega_t \Delta t$$

EKF: Jacobian G

- $G_t = \frac{\partial g}{\partial x} \big|_{\vec{\mu}_{t-1}}$ with state $\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$

$$g_1 = x_{t-1} + v_t * \cos(\theta_{t-1}) \Delta t$$

$$g_2 = y_{t-1} + v_t * \sin(\theta_{t-1}) \Delta t$$

$$g_3 = \theta_{t-1} + \omega_t \Delta t$$

$$\frac{\partial g_1}{\partial x} = 1$$

$$\frac{\partial g_1}{\partial y} = 0$$

$$\frac{\partial g_1}{\partial \theta} = -v_t * \sin(\theta_{t-1}) \Delta t$$

$$\frac{\partial g_2}{\partial x} = 0$$

$$\frac{\partial g_2}{\partial y} = 1$$

$$\frac{\partial g_2}{\partial \theta} = v_t * \cos(\theta_{t-1}) \Delta t$$

$$\frac{\partial g_3}{\partial x} = 0$$

$$\frac{\partial g_3}{\partial y} = 0$$

$$\frac{\partial g_3}{\partial \theta} = 1$$

EKF: Jacobian G

- $G_t = \frac{\partial g}{\partial x} \big|_{\vec{\mu}_{t-1}}$ with state $\vec{x}_t = \begin{bmatrix} x_t \\ y_t \\ \theta_t \end{bmatrix}$

$$G_t = \begin{bmatrix} 1 & 0 & -v_t * \sin(\theta_{t-1}) \Delta t \\ 0 & 1 & v_t * \cos(\theta_{t-1}) \Delta t \\ 0 & 0 & 1 \end{bmatrix}$$

EKF: The Algorithm

- After including and solving the (not so easy) equations from the Bayes Filter (Thrun, 2006, chapter 3.3.4) we get:

```
1:   Algorithm Extended_Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:        $\bar{\mu}_t = g(u_t, \mu_{t-1})$   
3:        $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$   
4:        $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$   
5:        $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$   
6:        $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$   
7:       return  $\mu_t, \Sigma_t$ 
```

Figure: Kalman Filter algorithm (Thrun, 2006)

EKF: The Algorithm

- After including and solving the (not so easy) equations from the Bayes Filter (Thrun, 2006, chapter 3.3.4) we get:

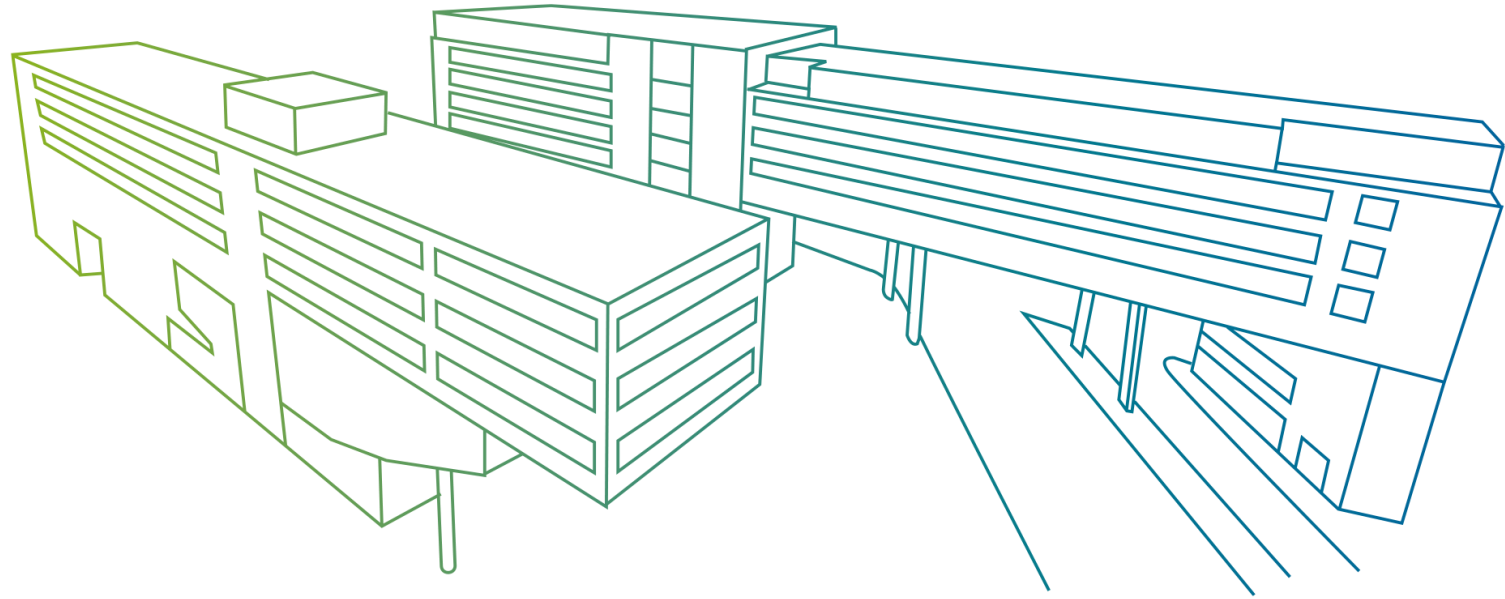
```
1:   Algorithm Extended_Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:        $\bar{\mu}_t = g(u_t, \mu_{t-1})$   
3:        $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$   
4:        $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$   
5:        $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$   
6:        $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$   
7:       return  $\mu_t, \Sigma_t$ 
```

Figure: Kalman Filter algorithm (Thrun, 2006)

Exercise: Extended Kalman Filter

- Implement an simple EKF for robot pose estimation
- Think of any nonlinear system with state $\vec{\mu}_t$
- Use a simple motion model and IMU measurements.
- Simulate some data (Motion and measurements)
- Implement an Extended Kalman filter
 - Prediction step (2 and 3)
 - Compute the Kalman Gain (4)
 - Implement the update step (5 and 6)
- Visualization of the true data, the estimated data and the IMU measurement
- Plot the Kalman Gain over time

```
1:  Algorithm Extended_Kalman_filter( $\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$ ):  
2:       $\bar{\mu}_t = g(u_t, \mu_{t-1})$   
3:       $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$   
4:       $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$   
5:       $\mu_t = \bar{\mu}_t + K_t (z_t - h(\bar{\mu}_t))$   
6:       $\Sigma_t = (I - K_t H_t) \bar{\Sigma}_t$   
7:      return  $\mu_t, \Sigma_t$ 
```



Probabilistic Robotics

Kalman Filter & Extended Kalman Filter

Lucas Muster, MSc